

Reading supplier invoices with a language model and checking them with code

Kalahari Components (Pty) Ltd, accounts payable, October 2025 to September 2026

The problem

A creditors clerk receives a supplier invoice and has to decide whether to pay it. Two things stand between the document and the payment. The invoice has to be read, and what it says has to be checked against what the company already knows: the purchase order that authorised the purchase and the goods received note that proves the goods arrived. That comparison is called the three way match.

Reading is the part that resists automation. Every supplier uses their own layout. One writes Invoice No., another writes Document No, a third writes INVOICE #. Totals sit in different corners of the page, amounts are formatted three different ways, and roughly one invoice in six arrives as a scan with no machine readable text on it at all. Checking, by contrast, is not difficult. It is arithmetic and comparison. What it demands is that the same facts always produce the same answer, and that the answer can be explained to an auditor and argued in front of a supplier.

The approach

The system splits those two jobs and gives them to different tools. A language model reads the document and returns structured data. Ordinary Python decides whether the document is acceptable. No model takes part in any control decision.

The split is deliberate, and getting it wrong is the usual failure of automation in finance. A language model is helpful by disposition. Asked to read an invoice where the VAT does not equal fifteen percent of the net amount, it will tend to report the figure that should be there rather than the figure that is. If it does that, the control looking for miscalculated VAT never fires, and the system reports a clean inbox. The instruction given to the model therefore says, in as many words, to transcribe and not to correct, and to report a subtotal only where one is printed rather than working one out from the lines.

Three routes run through the reading step, chosen per document on evidence rather than on a flag. An invoice with a usable text layer has that text sent to the model. An invoice without one has its page image sent instead, so the model reads it the way a person would. When no API key is available, a pattern matching parser takes over, so the project runs for anyone who downloads it. Where the total or the invoice number never came off the page, the invoice is marked for manual capture and the controls are not run on it at all, because findings raised against a document nobody could read describe the reader rather than the supplier.

The controls

Twelve checks run against every readable invoice. Each one produces the rand amount that would have been lost or mis-stated had the invoice been paid as it arrived, and a sentence telling the clerk what to do.

Code	What it checks	Severity
E01	Price above the purchase order	High
E02	Quantity billed exceeds quantity received	High
E03	Duplicate invoice	High
E04	No valid purchase order	Medium

E05	VAT not fifteen percent of the net amount	Medium
E06	Bank account differs from the master file	High
E07	Supplier not approved	High
E08	Invoice does not add up	Medium
E09	Invoice dated out of sequence	Medium
E10	Invoices split under an approval limit	High
E11	Not a valid tax invoice above R5 000	Medium
E12	Goods never booked in	High

Three of these are worth drawing out. E06 compares the bank account printed on the invoice against the account held on the supplier master file. A payment redirection attempt is arithmetically perfect, so it passes every check a person would perform by eye, and this is the only control that sees it. E10 looks across invoices rather than at one, and flags a run from a single supplier each sitting just below an approval limit inside a fortnight, which individually a junior may approve and together they may not. E11 tests the document against what a full tax invoice must contain. Above R5 000 a missing VAT registration number does not stop the goods being paid for, but it does stop the input tax being claimed, so the finding is valued at the VAT rather than at the whole invoice.

Results

Every error in the sample was planted deliberately before a single invoice was read, and written to a ground truth file. Accuracy is therefore measured rather than claimed. Recall is the share of planted errors the controls caught. Precision is the share of findings that were real.

Measure	Result
Invoices processed	284
Invoiced value checked	R10 589 521
Exposure found	R3 325 536
Invoices flagged	176
Passed every control	67
Sent for manual capture	41
Planted errors	148
Caught	119
Recall	80.4 percent
Precision	51.1 percent

Those figures come from the pattern matching route, with no language model involved, and they are reported rather than hidden because the comparison is the point. Reading accuracy across ten fields was 70.5 percent overall: 82.4 percent on invoices carrying a text layer, and 0.2 percent on scanned invoices. A regular expression cannot read a page with no text on it, so 41 invoices never reached the controls, and 24 of the 29 missed errors are simply documents the parser could not open. Of the alarms it did raise, roughly half were false, and almost all of those came from misreading a field rather than from anything being wrong with the invoice.

This is the clearest statement of what the model is actually buying. It is not better judgement, because judgement never left the Python. It is coverage. The controls cannot act on documents that were never read, and they waste a clerk's time when the fields they act on are wrong.

Limitations

The data is generated, and exceptions appear in roughly half of the invoices, far above a real creditors inbox. That rate was chosen so that every control has enough cases to be scored, and precision and recall on live data would differ. The system ends at the exception report: there is no approval workflow, no payment run and no link to an accounting package. Invoice lines are matched to order lines on stock code, so a supplier using their own codes would need matching on the description instead. The duplicate check compares the invoice number and the amount, and would not catch a resubmission with the amount altered by a rand.

What it demonstrates

The system takes unstructured documents, turns them into records a database can hold, applies business controls to those records, values the result in rands and reports it. It covers the ground between a document arriving and a decision being made, which is where most of the work in an information system actually sits. It also makes a choice about where a language model belongs, and can show the measurement that justifies the choice.